

A Heterogeneous Mapping of a Speech Neuroprosthesis Application

Iris Uwizeyimana, Oscar Sun, Natalie Enright Jerger

Edward S. Rogers Sr. Department of Electrical and Computer Engineering, University of Toronto
Toronto, Canada

iris.uwizeyimana@mail.utoronto.ca, jyoscar.sun@mail.utoronto.ca, natalie.enrightjerger@utoronto.ca

Abstract—Heterogeneous processors that combine CPUs, GPUs, and NPUs have become an important response to dark silicon and rising workload demands. The high performance per watt potential of these platforms offers a path to lower the energy consumption of ML applications, which is critical for the sustainable growth of these workloads. However, realizing these benefits requires understanding how different phases of an application interact with the compute and memory behavior of each hardware component. In this work, we use a machine-learning-based speech neuroprosthesis application to demonstrate how heterogeneous hardware can be leveraged to improve end-to-end performance. We introduce techniques for reducing memory usage, scheduling data movement, and managing synchronization, which enable different stages of the application to execute efficiently across the NPU, GPU, and multi-threaded CPU. Our evaluation shows a clear divergence in hardware preferences when optimizing for latency versus throughput per watt, and highlights the role of interference when deploying the application in a pipeline-parallel manner. This work shows that understanding both the application structure and the underlying hardware is essential for effectively exploiting heterogeneity and offers guidance for designing sustainable heterogeneous application mappings.

I. INTRODUCTION

The rise of machine-learning applications, coupled with the end of Dennard scaling, is driving a shift towards heterogeneous processors. As transistor dimensions shrink, leakage current prevents power from scaling down proportionally with the transistor size, resulting in power density increases. This increase in power density results in only a fraction of transistors being able to switch at high frequency without exceeding thermal and power limits; a phenomenon often referred to as dark silicon [20]. To sustain the high performance requirements of machine learning workloads under these constraints, vendors such as Apple [6]–[8], Intel [36], [37], and AMD [43] are integrating general-purpose CPUs with domain-specific accelerators such as GPUs and NPUs on the same die. Pairing high-throughput, reconfigurable accelerators with conventional von Neumann CPUs presents an opportunity to maximize performance per watt. The high performance per watt is essential not only for meeting computational demands, but also for reducing the energy footprint of ML workloads; providing a pathway for heterogeneous processors to be leveraged for lowering the carbon emissions of the ICT sector [15].

The ICT sector’s carbon emissions mostly comprise embodied carbon from manufacturing and operational carbon from use. The performance per watt improvement from heteroge-

neous processors offers an opportunity to reduce the net operational carbon. However, integrating multiple accelerators can increase embodied carbon from the increased chip area [18]. The remedy for this increased area is to amortize the added embodied carbon by maximizing the *useful* lifetime of all on-chip components while also minimizing energy consumption to reduce the operational carbon. Achieving this goal requires careful, application-aware considerations.

Using a speech neuroprosthesis workload, we highlight the importance of application understanding, profiling and refactoring for efficient heterogeneous processor mappings. These steps enable applications to fully leverage the diverse forms of parallelism, distinct memory hierarchies and specialized execution units of heterogeneous processors. The speech neuroprosthesis application is a brain-computer interface that converts neural signals into intended speech, providing a means of communication for patients with paralysis [51]. We select this workload for its tangible social impact and because it relies on widely used machine-learning building blocks, allowing insights from this work to generalize to a broad class of similar workloads. Importantly, the application offers a good example of a CPU-only workload that exhibits phases that can map across the CPU, GPU, and NPU with careful redesign.

The application follows an encoder-decoder architecture commonly used in natural language processing. The encoder consists of a Gated Recurrent Unit (GRU) model built from standard machine learning operations such as linear transformations and non-linear activation functions. The decoder consists of a Weighted Finite-State Transducer (WFST) graph traversal algorithm. More details of the application are covered in Sec. II-B. Each stage of the application exhibits distinct computational and memory behaviors. The GRU layers rely on large matrix multiplications with opportunities for data reuse, batching and tiling to hide the memory footprint, whereas the WFST decoder is dominated by irregular graph traversal with a large state space and limited temporal locality. These differences call for a carefully designed mapping that aligns each phase of the application with the strengths and limitations of the heterogeneous processor components.

We examine how to partition and map the full speech neuroprosthesis inference pipeline onto the AMD Strix Point processor [3]. The AMD Strix Point processor is a heterogeneous system-on-chip (SoC) that integrates AMD Zen5 CPU cores, an AMD RDNA™ 3.5 GPU, and an AMD XDNA™ 2

NPU, all sharing a unified DDR5 memory subsystem. We select this platform because it provides the three most commonly used components in modern heterogeneous processors (CPU, GPU and NPU) and offers sufficient on-chip memory to accommodate the application’s large memory footprint. The processor also offers an accessible programming interface for the NPU, a capability less readily available on other platforms.

Our work highlights several primitives required for efficient heterogeneous acceleration. These include applying Amdahl’s law to identify which stages to accelerate; reducing memory footprints using bit packing, quantization, and parameter tiling techniques; scheduling data movement to hide external-memory latency; and enforcing safe parallelism to avoid race conditions while minimizing synchronization overheads.

Our evaluation reveals that hardware preferences vary significantly depending on the optimization objective. For the GRU encoder, the GPU achieves the lowest latency, while the NPU provides the best throughput per watt due to its power efficiency. For the WFST decoder, the GPU again delivers the fastest latency, but multi-threaded CPU configurations yield better energy efficiency. Introducing pipeline parallelism across the two stages further improves performance. Interestingly, the hardware choices identified from optimizing each component in isolation differ from those observed under pipeline parallel execution. This is due to interference effects such as bandwidth contention, dynamic voltage and frequency scaling (DVFS) behavior, and accelerator time sharing.

Overall, our results demonstrate that achieving real-time or energy-efficient execution on heterogeneous processors requires careful, application-aware mapping that accounts not only for computational patterns but also for system-level interactions. By analyzing a realistic, multi-stage workload on a modern CPU–GPU–NPU platform, this work provides insights on offloading and parallelization strategies that can extend to a broad range of machine-learning algorithms to increase the efficient use of on-chip components.

II. BACKGROUND

This section begins with an overview of the AMD Strix Point processor and the used programming interfaces, followed by a description of the speech neuroprosthesis application and how we identify which application components to accelerate.

A. Hardware Architecture Description and NPU Programming Interface

The AMD Strix Point processor is an integrated heterogeneous SoC that combines AMD Zen5 CPU cores (12 in total), an AMD RDNA 3.5 GPU with 16 compute units, and an AMD XDNA 2 NPU, all of which access a common memory subsystem [2]. The memory subsystem uses $2 \times 2R$ DDR5-5600 [3], providing a theoretical memory bandwidth of 90 GB/s. Although the memory subsystem is shared, the CPU, GPU, and NPU do not maintain hardware memory coherence. Each Zen5 CPU core includes private L1 and L2 caches, and the cores communicate through a shared L3 cache. The

integrated GPU incorporates its own L2 cache while accessing the shared DDR5 memory.

The NPU features a coarse-grained reconfigurable array (CGRA)-like dataflow architecture built from a spatial array of compute tiles. The accelerator consists of a 4×8 array of compute tiles, a 1×8 array of memory tiles, and a 1×8 array of Shim tiles. Each compute tile integrates a scalar unit, a vector unit, L1 scratchpad memory, and DMA engines for explicitly scheduled data movement. The local L1 scratchpad holds 64KB of memory. The memory tiles act as the L2 scratchpad, with each tile providing 512KB of storage with their own DMA engines. While the compute and memory tiles include dedicated DMA engines for on-chip data movement, the NPU also provides an additional set of DMA engines, known as Shim tiles, which handle data transfers from external memory into either the memory tiles or directly into the compute-tiles.

Programming the NPU is supported through IRON, an open-source Python API designed for AMD Ryzen™ NPUs [31]. The NPU compute tiles are programmed by writing C++ kernels that use AMD’s AIE library to specify the vector or scalar computations. Using IRON, developers specify kernel-to-core mappings, data transfers within the NPU, and movement of data to/from external memory. The Xilinx Runtime library (XRT) enables the host program to interface with the NPU through the shared external memory [52]. GPU programming is done using AMD’s C++ GPU programming API, HIP (Heterogeneous-computing Interface for Portability) [4]. The GPU can also be programmed using higher lever frameworks such a PyTorch, which use HIP as the underlying backend.

B. Speech Neuroprosthesis Application Description

Fig. 1 provides a high-level overview of the speech neuroprosthesis application used in this study [51]. The end-to-end inference consists of four main steps: data pre-processing, GRU encoding, post-GRU processing and WFST decoding. In the pre-processing step, neural inputs are collected over a sliding time window and passed through a Gaussian smoother. The smoothed signals undergo a day-specific linear transformation. The signals are passed through a linear layer with specific per-day weight parameters to account for day-to-day variations in neural activity. This is followed by a softsign non-linear function and the necessary tensor reshaping before being forwarded to the GRU network (detailed in the Sec. II-C). The GRU step outputs a sequence of hidden states which the post-GRU step maps to phoneme¹ logits using a linear layer. Each output tensor of this step contains the predicted likelihoods over 42 possible phonemes at a single time step, and the full sequence of tensors represents these phoneme likelihoods across a time window. After further tensor reshapes, the post-GRU processing outputs are fed into the WFST model. The WFST model, described in Sec. II-D, uses the phoneme logits across time-steps to traverse a graph that transforms the logits into cohesive English sentences.

¹Phonemes are the smallest distinct sound units in a language that serve to differentiate spoken words (e.g., æ, ʌ, v).

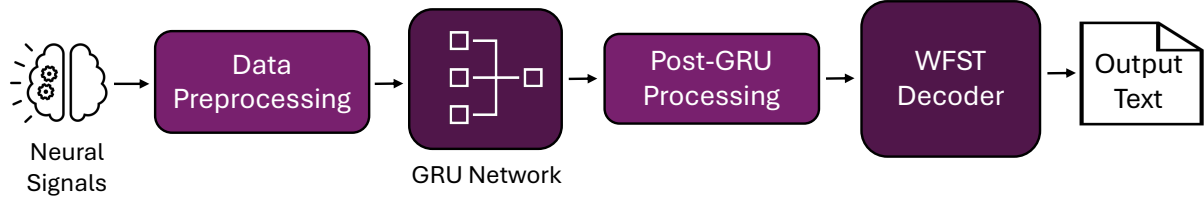


Fig. 1. High Level Speech Neuroprosthesis Application Description

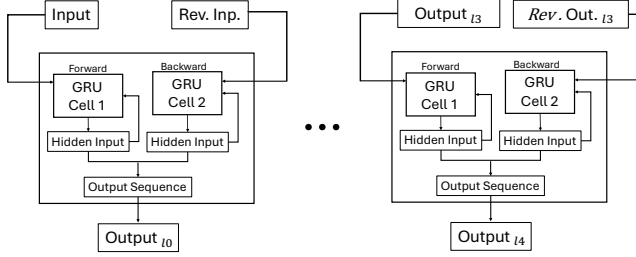


Fig. 2. GRU Model Architecture

Several works have sought to improve the accuracy of the speech neuroprosthesis application [9], [13], [21], [33]. While all these projects rely on a WFST decoding step to generate output sentences from phoneme logits over time, the first three steps of the application differ slightly across implementations. Most leaderboard-ranked systems² still apply Gaussian smoothing to the neural inputs and use an RNN (typically a bidirectional GRU) in the early processing stages.³ Therefore, we use the original implementation as it aligns closely to the state-of-the-art. There is currently no real-time deployment of the speech neuroprosthesis application, which creates an opportunity to explore batching different neural input sequences together for performance improvements.

C. GRU Network Topology

This work focuses on the original PyTorch implementation of the GRU model.⁴ Fig. 2 illustrates the model architecture. The model is a 5-layer bi-directional GRU, which effectively means there are 10 GRU cells in total: five cells form the forward layers (processing inputs from t_0 to t_n) while the other five form the backward layers (processing inputs from t_n to t_0). For a given layer, the forward and backward directions can be computed in parallel when the full sequence of t_0 to t_n is available. The processing of each sequence's time-step must happen sequentially due to the hidden input update; inter-layer communication must also happen sequentially as each layer consumes the outputs of the previous one.

Equation 1 summarizes the computation inside a single GRU cell. The GRU cell takes two inputs at each time step, the external input x_t and the hidden state from the previous time

step (i.e., the previous GRU output) h_{t-1} . The computation consists of six linear transformations (three applied to x_t and three to h_{t-1}) corresponding to the reset r , update z and candidate n gates, followed by non-linear (sigmoid, tanh) and element-wise operations (adds, multiplies).

$$\begin{aligned} r &= \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr}) \\ z &= \sigma(W_{iz}x_t + b_{iz} + W_{hz}h_{t-1} + b_{hz}) \\ n &= \tanh(W_{in}x_t + b_{in} + r \odot (W_{hn}h_{t-1} + b_{hn})) \\ h' &= (1 - z) \odot n + z \odot h_{t-1} \end{aligned} \quad (1)$$

W_{ir}, W_{iz}, W_{in} and b_{ir}, b_{iz}, b_{in} represent the weight matrices and bias vectors applied to the input x_t for the reset, update, and candidate gates, respectively. Similarly, W_{hr}, W_{hz}, W_{hn} and b_{hr}, b_{hz}, b_{hn} correspond to the weight matrices and biases applied to the previous hidden state h_{t-1} . The resulting hidden state h' represents the output of the current time step and is used as the hidden state input for the subsequent time step.

For the first layers (in both directions) the input dimension is 8192 and hidden size is 1024 while the rest of the layers have an input dimension of 2048 and a hidden size of 1024.

D. Understanding WFST Models Inference

Fig. 3 demonstrates a toy example of a WFST language model. The model is defined by a set of states and the arcs connecting them. Each arc represents a possible transition and is defined by four components: an input label, an output label, a weight, and the next state [35]. For example, in the toy WFST of Fig. 3, state 0 has three outgoing arcs. The input label is given by the first character, the output label appears inside the angle brackets, and the final number denotes the arc weight. Arc inputs can either be emitting (a character from the accepted inputs) or non-emitting (an empty input, denoted ϵ). Emitting arcs consume an input symbol while non-emitting arcs allow the model to transition between states without consuming a symbol. Non-emitting arcs are used to represent silent phonemes in a language or to allow transitions within the model that mark the beginning and end of utterances [27]. At each time step, the WFST receives a likelihood distribution over all plausible input phoneme characters. The model uses these logits to guide traversal of the finite state machine and generate a valid output sequence.

Decoding Algorithm: The speech neuroprosthesis application uses the WFST decoding algorithm from Kaldi, an open source toolkit widely used in speech recognition research [41].

²<https://eval.ai/web/challenges/challenge-page/2099/leaderboard/4944>

³The main exception is a very recent publication [21] that replaces the RNN with a lightweight transformer layer.

⁴https://github.com/cffan/neural_seq_decoder

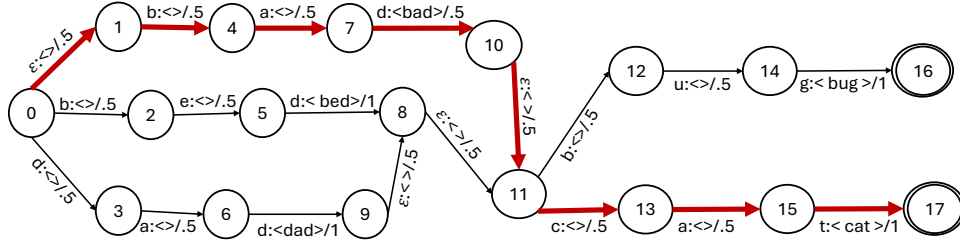


Fig. 3. WFST Toy Example. We use letters instead of phonemes to improve readability; the red arrows highlight a best path example.

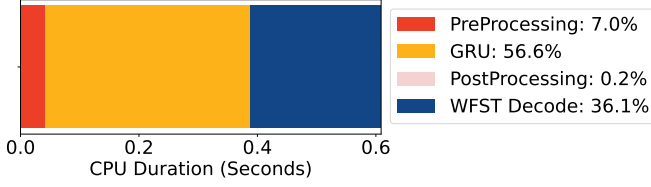


Fig. 4. Speech Neuroprosthesis execution time breakdown

Kaldi’s decoder implements a beam search algorithm [39] to traverse the WFST graph during inference. This beam search heuristic is derived from the breadth-first search (BFS) algorithm. Similar to BFS, beam-search expands the graph layer by layer, where each layer corresponds to nodes to be expanded at a certain time step (or frame). However, unlike standard BFS, beam search introduces a pruning mechanism that discards nodes whose accumulated cost exceeds a predefined threshold at each step. This pruning significantly reduces computational demands without sacrificing accuracy.

Using the toy example shown in Fig. 3, the first layer of nodes to be visited initially contains only state 0. During the first time-step, the search algorithm expands all arcs originating from state 0, activating states 1, 2 and 3, which then form the set of nodes to be visited in the next frame. When a node is expanded, the cost of each resulting child node is computed by adding the parent node’s cost, the arc’s weight and the negative phoneme logit for the arc’s input symbol at that time. Non-emitting arcs, which do not consume input symbols, omit the per-time-step logit term for cost computation.

After all expansions at a given time step are completed, beam pruning is applied. Once the algorithm has processed all time-step probabilities in the input sequence, it selects the final-state node with the lowest accumulated cost and performs a traceback to recover the node’s path and final output tokens.

E. Accelerating the Speech Neuroprosthesis Application

Fig. 4 show the single-threaded CPU execution runtime of the four application steps for one speech decoding pass. We exclude the model-parameter loading stage that happens at the very beginning since it is a one-time, non-recurring cost for long-running deployments. The GRU and WFST stages take up the majority of the time: the GRU accounts for 56.6% of the total runtime while the WFST decoding accounts for 36.1%. Therefore, guided by Amdahl’s law, we target these

two components for acceleration. The pre-processing and post-GRU processing parts are kept on the CPU.

III. ACCELERATING THE GRU

We explore three acceleration strategies for the GRU; mapping the model on the NPU, the GPU, or the multi-threaded CPU. Each mapping presents distinct trade-offs between latency and power efficiency especially as we vary the processed batch sizes. Because our implementation is based on PyTorch, deploying the model on the GPU simply involves specifying the GPU as the target device. The multi-threaded CPU version also only involves using the `set_num_threads` function to specify the thread count. However, mapping the model on the NPU, is more involved. This section explores how to map the GRU model on the XDNA-based NPU to exploit its spatial compute parallelism and on-chip memory hierarchy.

A. Architectural overview of GRU mapping on the NPU

Fig. 5 illustrates how a single GRU cell is mapped onto the NPU. The AMD XDNA-based NPU features a two-level interconnect hierarchy. The first and fastest level is an interconnect which links neighboring tiles in the north, south, east and west directions. This interconnect provides direct access to neighboring tiles’ memory [1]. The second level is a NoC-based interconnect that communicates through DMAs to transfer data across non-neighboring tiles. Our mapping strategy, shown in Fig. 5, is designed to minimize hop counts and maximize the use of the neighbor-to-neighbor communication. In our implementation, a single GRU cell occupies a 4×4 array of compute tiles. Since the AMD Strix Point NPU consists of a 4×8 compute-tile array, we can map two GRU cells on the NPU. This allows us to execute both the forward and backward cells of a GRU layer simultaneously.

B. NPU Data Movement

The most time-consuming aspect of NPU execution is the transfer of data from external memory. The techniques used to hide this data transfer latency depend heavily on the size of the model parameters. For layers that can fit entirely on the NPU, an optimal strategy to reduce external memory latency involves keeping the layer weights stationary in the device’s L2 memory and forming a feedback loop in which the output from each time step becomes the hidden input for the next. Although the initial bulk transfer of weights to L2 is expensive, the repeated reuse of the same weights across time steps amortizes

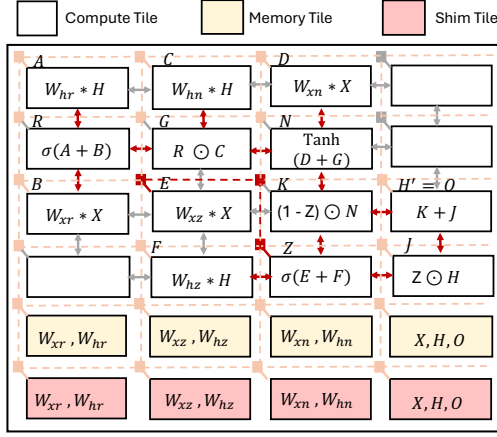


Fig. 5. Mapping of one GRU Cell on the NPU

this cost. Unfortunately, in our case, each GRU layer requires approximately 108MB of memory (for input dimension = 8192 and hidden dimension = 1024), which far exceeds the 4MB of available L2 memory on the NPU.

For layers whose parameters cannot fully fit on the NPU, a streaming-based approach combined with double buffering to overlap computation with data transfer is best. Instead of transferring the full layer parameters at once, we tile the parameters and choose the tile sizes such that the data transfer latency from external memory closely matches the computation latency, enabling near-complete overlap between the two. We achieve this communication-computation overlap using IRON's non-blocking memory access primitive `npu_dma_memcpy_nd`, which uses the Shim tiles to move data from external memory to the L2 memory tiles. We then use IRON's `ObjectFifo` primitive to transfer data between the memory and compute tiles as well as across compute tiles. All the memory hierarchies are double buffered to support the communication-computation overlap; hiding the external memory access latency. To further hide external memory latency, we batch the GRU inputs across multiple sequences since they share the same weights per layer. Batching across time steps is not possible because the GRU's hidden state is updated at each time step.

For each GRU cell, the inputs x_t and h_{t-1} have dimensions $(B \times S)$ and $(B \times H)$, respectively. Their corresponding weight matrices, W_x^* and W_h^* , have dimensions $(S \times H)$ and $(H \times H)$. We compute the tile sizes of x_t , h_{t-1} , W_x^* and W_h^* that would maximize L1 memory utilization, producing input tiles of size $(b \times s)$ and $(b \times h)$, and weight tiles of size $(s \times h)$ and $(h \times h)$. The resulting output tiles of size $(B \times H)$ are computed in a row-major order, yielding $(B/b) \times (H/h)$ total output tiles. During execution, DMAs stream one row of input tiles and one column of weight tiles to the corresponding compute tiles per iteration to compute a single output tile. For external memory access, data synchronization occurs only on the output buffer after each row of output tiles is computed. In each output-row iteration, a single input row and all corresponding weight tiles

are streamed in. Each iteration streams in the same weight tiles but uses the `npu_dma_memcpy_nd` offset parameter to advance to the next row of inputs.

On the host side, GRU computation is organized per layer and per time step. Each layer's weights are pre-allocated in host memory before execution. The layer weights are grouped into three buffers: $(W)_{(x,h)n}$, $(W)_{(x,h)z}$ and $(W)_{(x,h)r}$, each containing the parameters for both the forward and backward GRU cells. This grouping is required because the XRT runtime, used for host-NPU communication, supports only five I/O buffers per kernel invocation (including input and output buffers). For each layer, weights and input data are copied to the XRT I/O buffers using the host-side `memcpy`.

IV. ACCELERATING THE WFST DECODE

We consider mapping the WFST model either to a multi-threaded CPU or to a GPU, as the model's decoding process cannot fit within the NPU's on-chip memory. While the GRU's weight parameters also exceed the NPU's scratchpad memory capacity, their regular stride-based data access pattern enables the NPU's streaming dataflow architecture to hide external memory access latency by overlapping memory accesses with compute. In contrast, the irregular memory access patterns inherent to graph-based algorithms such as WFST decoding prevent efficient mapping to a streaming-dataflow architecture when the graph is too large to fit in on-chip memory.

Nonetheless, the arc expansions in the WFST model decode stage expose SIMD parallelism that make it a good candidate for GPU or multi-threaded CPU execution. Unfortunately, the WFST of our target application occupies a memory footprint that is too large to fit on the available GPU memory space. We apply bit packing and quantization strategies to reduce the model size sufficiently to fit on the GPU (Sec. IV-A). Sec. IV-B explores the GPU parallelization of the beam-search algorithm aimed at improving thread utilization and reducing synchronization overheads while ensuring correctness. Sec. IV-C presents the multi-threaded CPU implementation.

A. Reducing the WFST Memory Requirement

The original WFST model requires about 37GB of working memory. This memory size is too large to fit in the GPU memory space, even without taking into account additional memory required for GPU runtime overheads. Thankfully, the arcs of the model provide opportunities for quantization and bit packing for bitwidth reduction.

The arcs of the WFST model comprise an input, output, weight and next state parameter. In the baseline, all fields are 32-bits (ints or floats). Given our graph size (403,695,268 states), next state elements must remain 32-bit unsigned integers. The input/output label fields provide opportunity for bitwidth reduction. Since our model only accepts 42 possible input symbols, the input labels can fit in 6 bits. Given the number of possible output tokens, our output labels require a maximum of 17 bits to be fully represented. Since there are no 6 or 17 bit data precision types in conventional programming languages, we implement the input labels with a `uint8` type.

For the output label, we store its lower 16 bits in a uint16 type and carry the 17th bit in the most significant bit of the 8-bit input field.

During decoding, we recover the original values with a small amount of bit arithmetic as shown below:

- Get the original input: $\text{real_input} = \text{input} \& 0x7F$
- Extract 8th bit from the input: $\text{bit8} = (\text{input} \gg 7) \& 1$
- Reconstruct output: $\text{real_output} = \text{output} | (\text{bit8} \ll 16)$.

For the arc’s weight, after experimentation, we observe that quantizing the float32 bit weights to int16 and dequantizing the values during processing does not change the model’s graph traversal. We achieve this by using a uniform linear quantization [28], [47] where the quantized values are computed as:

$$w_{\text{quant}} = \text{round}\left(\frac{w - \min}{\max - \min} \times (2^{16} - 1)\right) \quad (2)$$

and dequantized during processing using the formula:

$$w_{\text{dequant}} = \left(\frac{w_{\text{quant}}}{2^{16} - 1} \times (\max - \min)\right) + \min \quad (3)$$

with max and min being the minimum and maximum values observed across all the arcs.

In our WFST model, max is 16.964844 while min is 0. This makes the quantization step size, $\Delta = (\max - \min) / (2^{16} - 1)$, equal to ~ 0.0002589 . Given Δ , the maximum observable error relative to the largest value is $\Delta / \max \approx 0.0015\%$. This error rate is too small to change the graph traversal’s accumulated cost enough to alter the chosen best path.

The quantization and bit-packing reduce the WFST working memory from 37GB to 19GB before extra GPU overheads and to 20GB after the overheads. We store the WFST arcs in a contiguous 1D array and maintain a state-offset table to track which arcs belong to each state. To reduce memory, the offset table only tracks the index of each state’s first arc, such that accessing the array by $[\text{state_number}]$ and $[\text{state_number}+1]$ gives the first and last index of the state’s arcs in the arcs array. This ensures both minimal space allocation and efficient $O(1)$ access.

B. GPU Mapping of the WFST Decoder

We adopt a GPU graph traversal algorithm similar to Enterprise [34] where a frontier queue tracks the nodes to be visited per layer and only nodes in this queue are expanded for the next layer’s computation. Mapping each node’s outgoing-arc expansions to a single thread introduces significant load imbalance due to the variability in the number of arcs per node. To increase thread utilization and reduce this imbalance, we incorporate the BFS thread-scheduling strategy proposed by Hong et al. [29]. Each node in the frontier queue is assigned its own thread block; using a grid-stride loop, the thread within each block fans out across the node’s outgoing arcs to compute traversal costs and update the frontier. The frontier queue is double-buffered to avoid race conditions and memory-contention overheads from concurrent reads and writes. Algorithm 1 gives the arc expansion pseudocode.

We implement a relaxed frontier-update mechanism on the GPU. An `atomicMin` operation is performed on a global

cost array that tracks the best cost of reaching each state in the current layer, and a thread appends to the next frontier only when its `atomicMin` succeeds (i.e., cost is the lowest observed so far). The frontier update is also done atomically using an `atomicAdd` to increment the next-frontier index before appending the state on the frontier and writing the state’s cost, parent state, and output label into the corresponding arrays. Because it is impossible to know in advance which states will be reached by the outgoing arcs, and because only one entry per state is allowed per layer, the cost array must be allocated with a size equal to the total number of states in the WFST (over 400 million in our case). In contrast, the frontier array along with the arrays storing the next-layer states, output labels and parent states only require a capacity equal to the beam size (10K entries in our setting), which is much smaller than the full state space.

After each expansion kernel finishes, the host constructs partial paths from the states in the next frontier, which are later used to extract the best path’s output token. During this path-construction loop, the host also swaps the frontier queues. To remove duplicates introduced by the relaxed frontier updates, the host only adds a state from the next frontier to the current frontier if its stored cost matches its cost array entry.

C. Multi-threaded CPU Mapping of the WFST Decoder

We implement the multi-threaded CPU WFST decoding using `pthread`s. Similar to the GPU implementation, we employ a double-buffered frontier queue to restrict graph traversal to nodes in the current layer. However, because the CPU provides far fewer hardware threads than the GPU, we parallelize over frontier nodes rather than over outgoing arcs.

Since there is no efficient `atomicMin` operation available on the CPU, we use mutex locks to synchronize updates to the cost array. Using a single mutex lock for the entire

Algorithm 1 GPU Arc Expansion Pseudocode

```

1: for all active nodes in parallel do
2:   // one thread block per node
3:    $\text{node\_idx} \leftarrow \text{blockIdx}$ 
4:    $\text{state} \leftarrow \text{active\_nodes}[\text{node\_idx}]$ 
5:    $\text{start} \leftarrow \text{state\_offsets}[\text{state}]$ 
6:    $\text{end} \leftarrow \text{state\_offsets}[\text{state} + 1]$ 
7:   for  $i \leftarrow \text{start} + \text{threadIdx}$  to  $\text{end}$  do
8:     // each thread processes one arc
9:      $c \leftarrow \text{compute\_arc\_cost}(\text{arcs}[i])$ 
10:     $\text{ns} \leftarrow \text{get\_next\_state}(\text{arcs}[i])$ 
11:    if  $c$  passes pruning cutoff then
12:       $\text{old\_cost} \leftarrow \text{atomicMin}(\text{Costs}[\text{ns}], c)$ 
13:      if  $c < \text{old\_cost}$  then
14:        // Atomic swap success
15:         $\text{update\_next\_frontier}(\text{ns}, c, \dots)$ 
16:      end if
17:    end if
18:  end for
19: end for

```

cost array results in unnecessary lock contention between arcs that update different next states. However, assigning one lock per state would require excessive memory given the large state space. To address this trade-off, we create a small vector of mutex locks (1024 in total) and use a modulo operation (`next_state % 1024`) to determine which lock to acquire for each cost update. This approach provides a balance between reducing lock contention and minimizing the memory overhead associated with lock management.

V. EVALUATION

This section describes the methodology used for power measurement (Sec. V-A1) and for pipelining the speech neuroprosthesis application across two processing stages (Sec. V-A2). We then examine the latency, power, and energy characteristics of the various GRU and WFST mapping strategies (Sec. V-B), followed by an end-to-end evaluation of the application under different mappings and batch sizes (Sec. V-C).

A. Methodology

1) **Power Measurement:** While tools such as `rocm-smi` [5] and RAPL [32] provide access to GPU and CPU power measurements, the NPU power sensors are not exposed to the user. As a result, we use an external wall power monitor to estimate the average power consumption of the AMD Strix Point processor, both in idle and active states. The power monitor operates at a 60 Hz sampling frequency. Unfortunately, it is difficult to precisely synchronize a wall power monitor with the execution time of a process. To obtain reliable measurements, we run each process for enough iterations to produce at least three minutes of total runtime, ensuring synchronization and cold-start effects are averaged out.

2) **Pipelining application into two stages:** For the end-to-end evaluation, we divide the speech decoding pipeline into two stages. The first stage includes the preprocessing steps along with the GRU network inference. The second stage includes the post-GRU processing along with the WFST decoding. We consider mapping the GRU stage on the NPU, GPU, and multi-threaded CPU and the WFST stage on the GPU and multi-threaded CPU. These mappings can be executed in a pipelined manner; for the mapping where both the GRU and WFST run on GPU, pipeline parallelism can still be achieved through time multiplexing. The two stages are executed as separate processes that communicate using the POSIX TCP socket library.

After the GRU stage finishes processing a batch, it sends the WFST process messages containing the size of each sequence in the batch followed by the corresponding sequence vector data. Once all the sequences in a batch are sent, the GRU process transmits a final size of 0 to indicate the end of the transfer. The WFST process then decodes all received batches and returns to polling for new data from the client. In our evaluation, we vary the batch sizes from 1 to 128. For latency measurements, we record the execution time of both processes and take the higher value as the overall latency. The network transfer latency itself is not explicitly added under

the assumption that it should be overlapped with computation and therefore hidden. For power measurements, we record the system’s average power while both processes are running.

B. Per Component Latency Power and Energy Evaluation

1) **GRU evaluation:** Fig. 6(a) shows the latency of running the GRU model across the different tested mappings for batch sizes ranging from 1 to 128. For the multi-threaded CPU mappings, we sweep the CPU thread count from 1 (i.e., single-threaded) to 16 threads. When batching on the CPU, instead of concatenating all batches into a single input tensor of size $B \times S$ (B being the batch size and S the sequence length), we keep the batches as separate tensors and process them per loop. This was based on an observation where concatenating input batches for multi-threaded CPU processing results in worse performance, in terms of latency, compared to running the batches sequentially (even though PyTorch implements intra-operation parallelism).

An interesting observation from the latency results is the NPU’s performance across the different batch sizes. For a batch size of 1, the NPU exhibits the highest latency among all settings. This is primarily due to latency-limited external memory access from the DDR5 memory to the NPU’s local memory. Although our implementation overlaps communication with computation, the overlap is insufficient to hide memory-access latency at batch size 1. However, as the batch size increases, the NPU’s relative performance improves. The cost of memory accesses does not grow with the batch size because batching only increases the size of the input tensors which are small compared to the model weights. This confirms that the NPU is memory-bound for GRU processing, and batched execution is essential to hide memory latency.

Observation 1: The GRU-on-NPU mapping requires batching to overcome external memory access overheads.

The GPU, in contrast, does not face this issue. As shown in the figure, the GPU consistently achieves the best latency across batch sizes of 8 and up despite its relatively small cache size (2MB of L2). This behavior indicates that the GPU is more effective at hiding external memory access latency. Interestingly, using `.to(device)` to move the weight parameters from the host CPU to the GPU during computation instead of placing the tensors in the GPU memory space at initialization time results in a $2\times$ slowdown. Although the AMD Strix Point platform exposes a shared memory subsystem across the CPU, GPU, and NPU, PyTorch treats the CPU and GPU memory spaces as logically distinct. As a result, invoking `.to(device)` (with the GPU as the target device) triggers a `memcpy` operation, leading to the observed slowdown.

Observation 2: The GPU is better at hiding external memory accesses compared to the NPU, especially when large parameters are pre-allocated on the GPU memory space prior to execution.

Fig. 6(b) reports the average power consumption of the different mappings. The NPU is the most power-efficient

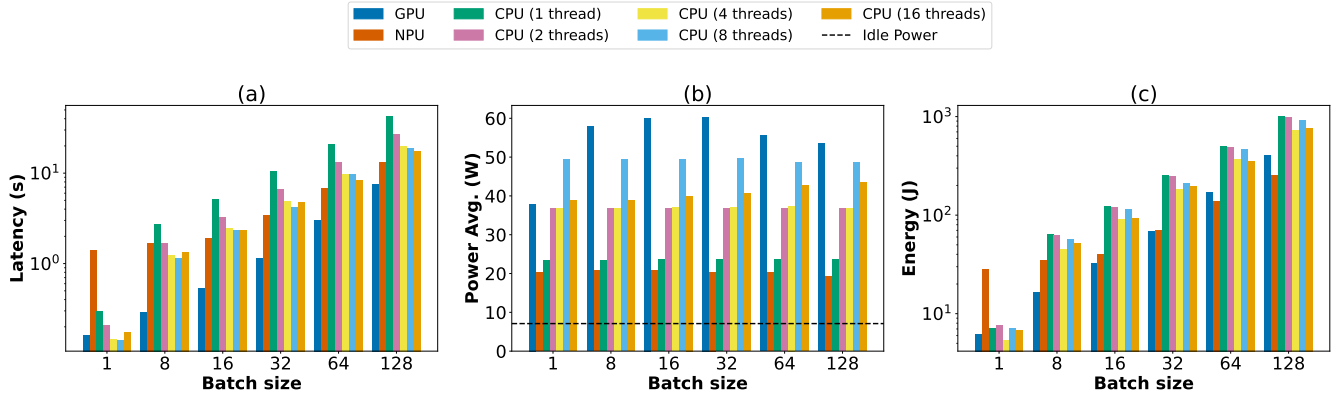


Fig. 6. GRU Evaluation: Latency (a), Power (b), and Energy (c) results.

across all tested settings, while the GPU draws the highest power. The multi-threaded CPU results show that power does not scale monotonically with thread count. For example, the 8-thread configuration consumes more power than the 16-thread configuration. Further profiling shows that the 8-thread mapping operates at a higher frequency, while DVFS restricts the frequency more aggressively when using 16 threads. Specifically, profiling with *htop* shows that under the 8-thread setting, all active threads run at $\sim 3.5GHz$; whereas, in the 16-thread setting, only five threads reach $\sim 3.7GHz$ while the remaining eleven are throttled to $\sim 2.3GHz$.

Observation 3: Due to DVFS behavior, multi-threaded CPU configurations with fewer threads run at higher operating frequencies. As a result, these configurations can sometimes consume more power than those with more threads.

Fig. 6(c) presents the total energy consumption across all batch sizes. Although the NPU does not provide the best latency, it delivers the lowest energy for batch sizes of 32 and above thanks to its low power usage. The GPU achieves the best energy for batch sizes of 8 and 16 while a 4-thread CPU mapping obtains the lowest energy at a batch size of 1. These results highlight the fine-grained trade-off between power and latency when optimizing for energy efficiency.

Observation 4: Optimizing for energy efficiency exposes a trade-off between latency and power; while latency-efficient mappings determine energy-efficiency at small batch sizes, power plays a more important role as the batch sizes increase.

2) **WFST evaluation:** Fig. 7(a) shows the latency of the WFST decoding algorithm across the evaluated mappings. We only present results for a batch size of 1 because WFST batches must be processed sequentially. Each batch requires a large cost array for correctness, and this memory footprint becomes prohibitive when attempting multi-batch execution. Across all tested thread counts, the GPU achieves lower decoding latency than the CPU. An interesting trend on the CPU is that increasing the thread count from 1 to 4 reduces latency, but performance degrades again when using 8 or 16

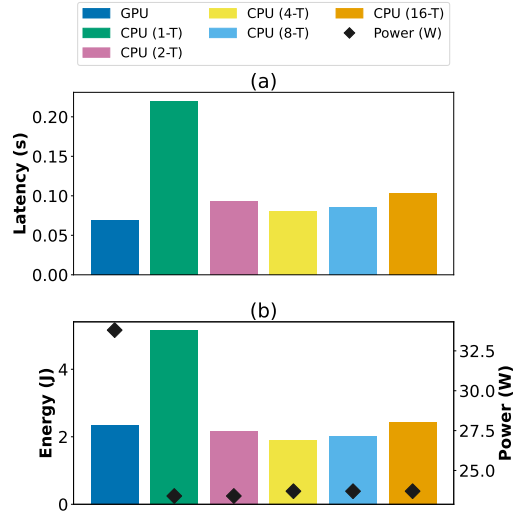


Fig. 7. WFST Evaluation Results: (a) shows the latency while (b) shows the power and energy results

threads. This reflects the trade-off between parallelism benefits versus thread-creation and data-partitioning overheads.

Observation 5: CPU thread creation and scheduling overheads can negatively impact application performance.

Fig. 7(b) presents the energy and power measurements for WFST decoding. Among the CPU configurations, the 4-thread mapping yields the lowest energy consumption. Although the GPU provides the best latency, its high power draw places it fourth in energy efficiency. Overall, the power consumption of both the GPU and CPU during WFST decoding is significantly lower than during GRU inference. This suggests that WFST decoding is primarily memory-bound, which is consistent with its larger memory footprint.

C. End-to-End Evaluation

Fig. 8 presents the end-to-end evaluation results. From the latency measurements, a key observation is that for batched

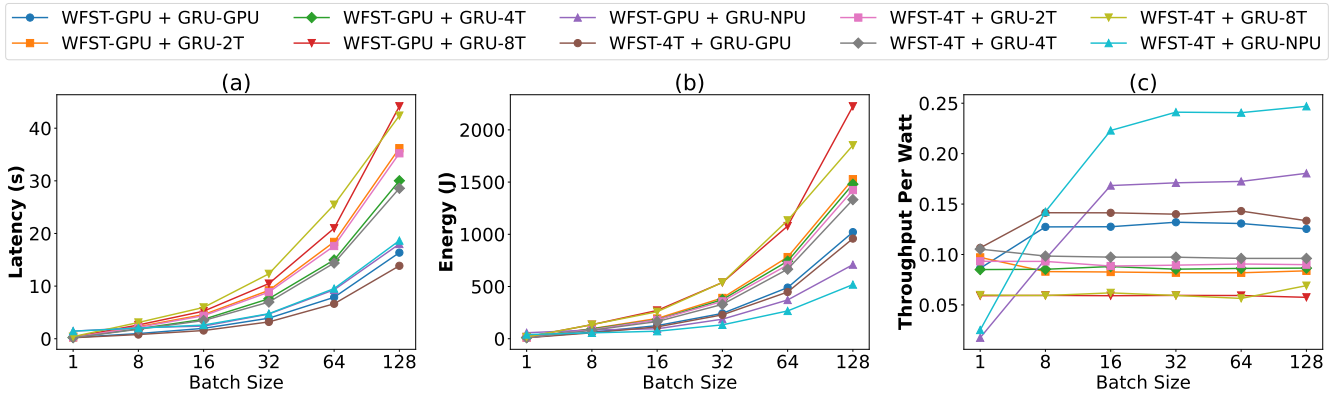


Fig. 8. End-to-End Evaluation: Latency (a), Energy (b), and Throughput Per Watt (c) results.

processing, multi-threaded CPU mappings of the GRU consistently result in higher overall latency. As observed earlier, this behavior can be attributed to DVFS effects where increasing the number of CPU threads reduces the operating frequency, negatively impacting performance. Interestingly, we observe CPU power throttling effects even in settings where the WFST decoding was mapped to the GPU. Upon further investigations, we find this is not due to the GPU being throttled but from the throttling of CPU threads created for GPU kernel dispatching. We observe the GPU decoder process spawns multiple CPU threads although the host-side computation was single-threaded; indicating the HIP runtime launches CPU threads to handle kernel dispatches.

Observation 6: DVFS-induced interferences affect multi-threaded CPU mappings during pipeline execution. They also impact GPU processes by throttling the CPU threads responsible for GPU kernel dispatch.

Another source of interference we identify in the pipeline-parallel configurations is memory access contention. Using `perf` to monitor the unified memory controllers data slot clock, we observe up to an $8\times$ increase in read/write cycles across all four memory controllers when comparing pipeline-parallel execution to sequential runs. These memory accesses are evenly distributed across the controllers.

Observation 7: Memory contention effects are accentuated in pipeline processing.

Among the evaluated configurations, the GRU-GPU mappings achieve the lowest latency with WFST-4T+GRU-GPU having the lowest latency overall. From the latency results, we observe that per component optimization in isolation differs from the end-to-end optimization when dealing with pipeline parallelism. This is sometimes due to the DVFS effects mentioned earlier. For example, while the 8-thread CPU configuration achieved the lowest latency for GRU processing in isolation at batch size 1, the GPU mapping of the GRU achieved the best end-to-end mapping. Hardware time sharing

overheads can also contribute to this effect. For instance, the WFST-4T+GRU-GPU achieves better latency than the WFST-GPU+GRU-GPU although the WFST-GPU is the most latency efficient mapping in isolation.

Observation 8: Per-component hardware preferences differ when running processes in isolation vs. in a pipelined execution.

Looking at energy results, we again observe that latency optimal mappings differ from energy optimal mappings with the WFST-4T+GRU-NPU achieving the best energy efficiency overall. This is further confirmed in the throughput per watt results. We compute throughput per watt as the ratio of the batch size and corresponding energy consumption. An interesting observation is that increasing the batch size for the multi-threaded CPU GRU mappings does not improve throughput (and therefore does not improve throughput per watt). This arises because PyTorch parallelism is applied per tensor over the operations, rather than across tensors; thus, larger batch sizes do not translate into higher effective throughput for these configurations. The GRU-GPU mappings also exhibit relatively flat throughput-per-watt curves. This is primarily because the WFST phase, which is computed sequentially, is the latency dominant for most batch sizes (except at batch sizes 1 and 128). As a result, increasing parallelism in the GRU stage does not meaningfully influence end-to-end throughput.

Overall, this evaluation highlights that the preferred mapping for each application component depends strongly on the optimization objective. For minimizing end-to-end latency, the pipelined **WFST-4T + GRU-GPU** configuration is the most effective, achieving a latency of 0.189s at batch size 1, an improvement over the original 0.6s implementation that is within the 200ms BCI processing threshold [19]. In contrast, when optimizing for carbon-aware energy efficiency, the **WFST-4T + GRU-NPU** mapping achieves the highest throughput per watt at batch sizes of 8 and above. These results show that latency optimal mappings do not necessarily align with energy or carbon efficient ones. Energy optimal configurations favor batched instead of real-time processing

due to the slow memory access latency of accelerators. Finally, while naively using all on-chip components can hurt performance, each component offers distinct advantages that should be considered when mapping applications to improve overall processor usage.

D. Generalizing our findings beyond the Speech Neuroprosthesis application

While this work focuses on a speech neuroprosthesis application, the insights from our evaluation extend to a broader class of machine learning workloads. The computational patterns observed in the GRU model closely resemble those found in many modern DNN architectures. For example, transformer-based models rely heavily on matrix-matrix multiplications to implement linear projections for the query, key and value tensors as well as the attention computation. Furthermore, similar to GRU networks, these models also employ element-wise operations including multiplications for scaling, additions for residual connections and non-linear activation functions [48].

Importantly, state-of-the-art DNN models are memory-intensive. Our evaluation highlights that, for memory-bound workloads, it is essential to account for the overheads associated with data movement between the host CPU and accelerator devices. In these scenarios, communication rather than compute throughput becomes the dominant performance factor. We show that effectively hiding I/O-related overheads is critical not only for latency optimizations but also for improving performance per watt. For example, although the NPU exhibits the lowest raw power consumption among the 3 device types, it observes the poorest performance per watt when batching is not employed. This is because batching is essential to amortize the external memory access latency.

Finally, while the bit-packing techniques discussed for the graph’s input and output labels are specific to WFST models, (which are themselves widely used and have applications beyond speech recognition, including speculative decoding in Transformer-based LLMs [38] or Markov models with input-dependent state transitions that emit an output per transition), the quantization of graph edges weights can be leveraged by other graph-based algorithms with weighted edges, such as single-source shortest path algorithms.

VI. RELATED WORK

DNN Acceleration/Optimization: Early accelerator designs such as Eyeriss [12] and Dadiannao [14] employ fixed dataflow mappings (e.g., weight or row stationary strategies) to improve data reuse across CNNs. Tools such as Timeloop [40], CoSA [30] and LLMCompass [54] provide design space exploration frameworks that search for optimal mapping and tiling strategies to minimize latency given an accelerator and model architecture. Multi-chip module accelerators [11], [25], [26], [45] highlight the need to account for on-chip network contentions when mapping DNNs, while works such as Tetris and Plasticine [25], [42] emphasize the importance memory hierarchy optimizations play in efficient DNN mapping. Rosti

et al. [44] explore using AMD’s XDNA NPUs to accelerate GPT-2 training.

Mapping graph traversal algorithms on GPUs: Prior studies explore mapping WFST decoding to GPUs [10], [24]. Similar to our mapping, these works apply parallelism across the graph’s outgoing arcs. Although not specific to WFST, Frohner et al. [23] propose a layer-by-layer parallelization of the beam search algorithm across nodes within each layer. Other works addressing graph traversal on GPUs, such as those implementing the SSSP algorithm [16], [49], [50], employ complementary techniques that we also leverage, including double buffering to prevent race conditions and fixed-size arrays to reduce dynamic memory allocation costs.

Quantization: A wide range of research has focused on quantizing ML models from high-precision formats such as FP32 to lower-precision representations, including 16, 8, and 4-bit formats. Some works apply quantization to improve computational efficiency by exploiting integer matrix–matrix multiplication units [17], [53], while others use quantization for memory compression, decompressing the quantized values on-the-fly during computation [22], [46], [47].

VII. CONCLUSION

This work demonstrates how a multi-stage speech neuroprosthesis application can be efficiently mapped onto heterogeneous CPU-GPU-NPU systems by aligning each stage with the hardware component best suited to its computational and memory behavior. Using the AMD Strix Point Processor, we demonstrate that the GRU and WFST stages favor different hardware, depending on whether latency or energy is the primary objective, and that pipeline-level execution introduces cross-component interference effects not visible in isolated measurements. These findings highlight that real-time or energy-efficient execution on heterogeneous processors requires application-specific restructuring, careful data-movement scheduling, and system-level coordination. Looking ahead, we hope insights from this study can be leveraged by other works to efficiently map applications across heterogeneous processors. Furthermore, we hope our insights can inform future heterogeneous processor designs, particularly in guiding how memory hierarchies, component sharing and software runtimes can evolve to better support multi-phase workloads with diverse execution patterns.

ACKNOWLEDGEMENTS

We thank the reviewers for their helpful feedback. We are grateful to Victor Kariofillis and Raghavendra Pothukuchi for introducing us to the speech neuroprosthesis application. We also thank members of the NEJ research group, as well as Mehdi Saeedi and Eddie Richter from AMD, for their helpful comments and support. This work was supported by equipment donations from AMD and funding from the NSERC Alliance program (ALLRP 591075-2023), the NSERC CREATE program on Sustainable Data Systems for Data Science (CREATE 584767-2024), and the Canada Research Chair program.

REFERENCES

- [1] Advanced Micro Devices, Inc., *AI Engine Tile Architecture*, AMD, 2023, versal Adaptive SoC AI Engine Architecture Manual (AM009, Revision 1.3). [Online]. Available: <https://docs.amd.com/r/en-US/am009-versal-ai-engine/AI-Engine-Tile-Architecture>
- [2] Advanced Micro Devices, Inc., *AMD Unveils Next-Gen “Zen 5” Ryzen Processors to Power Advanced AI Experiences*, AMD, 2024. [Online]. Available: <https://www.amd.com/en/newsroom/press-releases/2024-6-2-amd-unveils-next-gen-zen-5-ryzen-processors-to-p.html>
- [3] Advanced Micro Devices, Inc., “AMD Ryzen™ AI 9 HX 370,” <https://www.amd.com/en/products/processors/laptop/ryzen/ai-300-series/amd-ryzen-ai-9-hx-370.html>, 2025, accessed: 2025-12-11.
- [4] Advanced Micro Devices, Inc., *HIP Documentation (ROCm 6.4.3)*, AMD ROCm Documentation, 2025. [Online]. Available: <https://rocm.docs.amd.com/projects/HIP/en/docs-6.4.3/>
- [5] Advanced Micro Devices, Inc., *ROCm System Management Interface (ROCm SMI) Library Documentation*, AMD, 2026. [Online]. Available: https://rocm.docs.amd.com/projects/rocm_smi_lib/en/latest/
- [6] Apple, “Apple unleashes m1 — the first system-on-a-chip for mac,” <https://www.apple.com/ca/newsroom/2020/11/apple-unleashes-m1/>, Nov. 2020, press release.
- [7] Apple, “Apple unveils m2, taking the breakthrough performance and capabilities of m1 even further,” <https://www.apple.com/ca/newsroom/2022/06/apple-unveils-m2-with-breakthrough-performance-and-capabilities/>, Jun. 2022, press release.
- [8] Apple, “Apple unveils m3, m3 pro, and m3 max, the most advanced chips for a personal computer,” <https://www.apple.com/ca/newsroom/2023/10/apple-unveils-m3-m3-pro-and-m3-max-the-most-advanced-chips-for-a-personal-computer/>, Oct. 2023, press release.
- [9] T. Benster, G. Wilson, R. Elisha, F. R. Willett, and S. Druckmann, “A cross-modal approach to silent speech with LLM-enhanced recognition,” *arXiv preprint arXiv:2403.05583*, 2024.
- [10] H. Braun, J. Luitjens, R. Leary, T. Kaldewey, and D. Povey, “GPU-accelerated Viterbi exact lattice decoder for batched online and offline speech recognition,” in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 7874–7878.
- [11] J. Cai, Z. Wu, S. Peng, Y. Wei, Z. Tan, G. Shi, M. Gao, and K. Ma, “Gemini: Mapping and architecture co-exploration for large-scale DNN chiplet accelerators,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 156–171.
- [12] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE journal of solid-state circuits*, vol. 52, no. 1, pp. 127–138, 2016.
- [13] Y. Chen and X. Zheng, “speechBCI_2024: Second-place solution for the Brain-to-Text Benchmark ‘24,” https://github.com/CIBR-Okubo-Lab/speechBCI_2024, 2024, source code repository.
- [14] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun *et al.*, “Dadiannao: A machine-learning supercomputer,” in *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2014, pp. 609–622.
- [15] P. Dangi, T. K. Bandara, S. Sheikhpour, T. Mitra, and L. Eeckhout, “Sustainable hardware specialization,” in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–9.
- [16] A. Davidson, S. Baxter, M. Garland, and J. D. Owens, “Work-efficient parallel GPU methods for single-source shortest paths,” in *2014 IEEE 28th International Parallel and Distributed Processing Symposium*. IEEE, 2014, pp. 349–359.
- [17] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale,” *Advances in neural information processing systems*, vol. 35, pp. 30 318–30 332, 2022.
- [18] L. Eeckhout, “Focal: A first-order carbon model to assess processor sustainability,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 401–415.
- [19] G. Eichler, L. Piccolboni, D. Giri, and L. P. Carloni, “MasterMind: Many-accelerator SoC architecture for real-time brain-computer interfaces,” in *2021 IEEE 39th International Conference on Computer Design (ICCD)*. IEEE, 2021, pp. 101–108.
- [20] H. Esmailzadeh, E. Blem, R. St. Amant, K. Sankaralingam, and D. Burger, “Dark silicon and the end of multicore scaling,” in *Proceedings of the 38th annual International Symposium on Computer Architecture*, 2011, pp. 365–376.
- [21] E. Feghhi, S. Kaasyap, N. Hadidi, and J. C. Kao, “Time-masked transformers with lightweight test-time adaptation for neural speech decoding,” *arXiv preprint arXiv:2507.02800*, 2025.
- [22] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” *arXiv preprint arXiv:2210.17323*, 2022.
- [23] N. Frohner, J. Gmys, N. Melab, G. Raidl, and E.-G. Talbi, “Parallel beam search for combinatorial optimization,” in *Workshop Proceedings of the 51st International Conference on Parallel Processing*, 2022, pp. 1–8.
- [24] D. Galvez and T. Kaldewey, “GPU-accelerated WFST beam search decoder for CTC-based speech recognition,” in *2023 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2023, pp. 1–7.
- [25] M. Gao, J. Pu, X. Yang, M. Horowitz, and C. Kozyrakis, “Tetris: Scalable and efficient neural network acceleration with 3D memory,” in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017, pp. 751–764.
- [26] M. Gao, X. Yang, J. Pu, M. Horowitz, and C. Kozyrakis, “Tangram: Optimized coarse-grained dataflow for scalable NN accelerators,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 807–820.
- [27] P. N. Garner, “Silence models in weighted finite-state transducers,” in *INTER_SPEECH*, 2008, pp. 1817–1820.
- [28] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, “Deep learning with limited numerical precision,” in *International conference on machine learning*. PMLR, 2015, pp. 1737–1746.
- [29] S. Hong, S. K. Kim, T. Oguntebi, and K. Olukotun, “Accelerating CUDA graph algorithms at maximum warp,” *Acm Sigplan Notices*, vol. 46, no. 8, pp. 267–276, 2011.
- [30] Q. Huang, M. Kang, G. Dinh, T. Norell, A. Kalaiah, J. Demmel, J. Wawrzynnek, and Y. S. Shao, “CoSA: Scheduling by constrained optimization for spatial accelerators,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 554–566.
- [31] E. Hunhoff, J. Melber, K. Denolf, A. Bisca, S. Bayliss, S. Neuendorffer, J. Fifield, J. Lo, P. Vasireddy, P. James-Roxby, and E. Keller, “Efficiency, Expressivity, and Extensibility in a Close-to-Metal NPU Programming Interface,” in *2025 IEEE 33rd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2025, pp. 85–94.
- [32] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, “RAPL in action: Experiences in using RAPL for power measurements,” *ACM Transactions on Modeling and Performance Evaluation of Computing Systems (TOMPECS)*, vol. 3, no. 2, pp. 1–26, 2018.
- [33] J. Li, T. Le, C. Fan, M. Chen, and E. Shlizerman, “Brain-to-text decoding with context-aware neural representations and large language models,” *Journal of Neural Engineering*, 2025.
- [34] H. Liu and H. H. Huang, “Enterprise: breadth-first graph traversal on GPUs,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2015, pp. 1–12.
- [35] M. Mohri, F. Pereira, and M. Riley, “Weighted finite-state transducers in speech recognition,” *Computer Speech & Language*, vol. 16, no. 1, pp. 69–88, 2002.
- [36] A. O. Munch, N. Nassif, C. L. Molnar, J. Crop, R. Gammack, C. P. Joshi, G. Zelic, K. Munshi, M. Huang, C. R. Morganti, S. Kandula, and A. Biswas, “Emerald Rapids: 5th-generation Intel® Xeon® scalable processors,” in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67. IEEE, 2024, pp. 40–42.
- [37] N. Nassif, A. O. Munch, C. L. Molnar, G. Pasdast, S. V. Lye, Z. Yang, O. Mendoza, M. Huddart, S. Venkataraman, S. Kandula, R. Marom, A. M. Kern, B. Bowhill, D. R. Mulvihill, S. Nimmagadda, V. Kalidindi, J. Krause, M. M. Haq, R. Sharma, and K. Duda, “Sapphire Rapids: The next-generation Intel Xeon scalable processor,” in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 44–46.
- [38] T. D. Nguyen, J.-H. Kim, J. Choi, S. Choi, J. Park, Y. Lee, and J. S. Chung, “Accelerating codec-based speech synthesis with multi-token prediction and speculative decoding,” in *ICASSP 2025-2025 IEEE*

International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2025, pp. 1–5.

- [39] P. S. Ow and T. E. Morton, “Filtered beam search in scheduling,” *The International Journal Of Production Research*, vol. 26, no. 1, pp. 35–62, 1988.
- [40] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, “Timeloop: A systematic approach to DNN accelerator evaluation,” in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2019, pp. 304–315.
- [41] D. Povey, A. Ghoshal, G. Boulianne, L. Burget, O. Glembek, N. Goel, M. Hannemann, P. Motlicek, Y. Qian, P. Schwarz, J. Silovsky, G. Stemmer, and K. Vesely, “The kaldi speech recognition toolkit,” in *IEEE 2011 workshop on automatic speech recognition and understanding*, vol. 1. Hawaii, 2011, pp. 5–1.
- [42] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, “Plasticine: A reconfigurable architecture for parallel patterns,” *ACM SIGARCH Computer Architecture News*, vol. 45, no. 2, pp. 389–402, 2017.
- [43] A. Rico, S. Pareek, J. Cabezas, D. Clarke, B. Ozgul, F. Barat, Y. Fu, S. Münz, D. Stuart, P. Schlangen, P. Duarte, S. Date, I. Paul, J. Weng, S. Santan, V. Kathail, A. Sirasao, and J. Noguera, “AMD XDNA™ NPU in Ryzen™ AI processors,” *IEEE Micro*, 2024.
- [44] A. Rösti and M. Franz, “Unlocking the AMD neural processing unit for ML training on the client using bare-metal-programming tools,” in *2025 IEEE 33rd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2025, pp. 271–271.
- [45] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, S. G. Tell, Y. Zhang, W. J. Dally, J. S. Emer, C. T. Gray, B. K. Khailany, and S. W. Keckler, “Simba: Scaling deep-learning inference with multi-chip-module-based architecture,” in *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*, 2019, pp. 14–27.
- [46] S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, “Q-bert: Hessian based ultra low precision quantization of bert,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 8815–8821.
- [47] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: High-throughput generative inference of large language models with a single GPU,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.
- [48] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [49] K. Wang, D. Fussell, and C. Lin, “A fast work-efficient SSSP algorithm for GPUs,” in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 133–146.
- [50] Y. Wang, A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens, “Gunrock: A high-performance graph processing library on the GPU,” in *Proceedings of the 21st ACM SIGPLAN symposium on principles and practice of parallel programming*, 2016, pp. 1–12.
- [51] F. R. Willett, E. M. Kunz, C. Fan, D. T. Avansino, G. H. Wilson, E. Y. Choi, F. Kamdar, M. F. Glasser, L. R. Hochberg, S. Druckmann, K. V. Shenoy, and J. M. Henderson, “A high-performance speech neuroprosthesis,” *Nature*, vol. 620, no. 7976, pp. 1031–1036, 2023. [Online]. Available: <https://github.com/fwillett/speechBCI>
- [52] Xilinx Inc., “mlir-aie programming guide,” https://github.com/Xilinx/mlir-aie/tree/main/programming_guide, 2025, accessed: 2025-11-08.
- [53] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, “Zeroquant: Efficient and affordable post-training quantization for large-scale transformers,” *Advances in neural information processing systems*, vol. 35, pp. 27 168–27 183, 2022.
- [54] H. Zhang, A. Ning, R. B. Prabhakar, and D. Wentzlaff, “LLMCompass: Enabling efficient hardware design for large language model inference,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1080–1096.